Object Oriented Programming (CSEG1044)
Unit 05 – Lecture 02 Notes
Set/SortedSet + Map/SortedMap

April 1, 2026

Contents

1 Lecture Snapshot **2**

2 Lecture Overview **2**

3 Prerequisite Bridge **2**

4 Learning Outcomes **2**

5 Suggested Reading Order **3**

6 Key Terms (Quick) **3**

7 Detailed Notes **3**

7.1 Set vs List; HashSet vs TreeSet 3

7.2 Equality and hashing basics (must know) 4

7.3 Map basics; HashMap vs TreeMap 4

8 Running Project Connection **4**

9 Common Mistakes and Confusions **5**

10 Worked Example: set + map counting **5**

11 Demo / Observation Task **5**

12 Practice Check (with brief solutions) **6**

13 Self-Study Practice (no solutions) **6**

14 Further Study **6**

15 Exit Questions (with sample answers) **6**

1 Lecture Snapshot

- **Unit focus:** Collections Framework and Wrapper Classes
- **Main new ideas:** Set vs List; HashSet vs TreeSet, Equality and hashing basics (must know), Map basics; HashMap vs TreeMap
- **Course thread:** Use Set/SortedSet + Map/SortedMap to manage in-memory project data more cleanly and predictably inside Campus Resource Manager.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

2 Lecture Overview

This lecture focuses on **uniqueness** (Set) and **key-value lookup** (Map). These structures are essential for problems like:

- removing duplicates,
- membership checks ("is this item present?"),
- counting and grouping (frequency tables),
- fast search by key (roll number → student details).

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

3 Prerequisite Bridge

Before reading the new material in depth, do a fast recall of the older ideas listed below.

- **List basics**: Sets and maps extend the collection story by optimizing for uniqueness and key-based lookup. Main reference: `Unit 05 / Lecture 01 slides/notes`.
- **Class design**: Set and map correctness depends on stable equality logic for stored objects. Main reference: `Unit 02 / Lecture 01 slides/notes`.

This lecture only revisits those ideas briefly so that the main explanation can stay focused on the new concept sequence.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

4 Learning Outcomes

After this lecture, you should be able to:

- Choose Set vs List and `HashSet` vs `TreeSet`.
- Explain `equals`/`hashCode` and common mistakes.
- Use `HashMap`/`TreeMap` for counting and sorted keys.

5 Suggested Reading Order

1. Read the **Prerequisite Bridge** first and confirm each recalled idea.
2. Study the central explanation in this order: *Set vs List; HashSet vs TreeSet → Equality and hashing basics (must know) → Map basics; HashMap vs TreeMap.*
3. Trace the worked example line by line before checking short answers.
4. Attempt the practice section without looking at the solution immediately.
5. Finish with the exit questions and further-study suggestions to confirm retention.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

6 Key Terms (Quick)

- **Set**: collection with no duplicates.
- **SortedSet**: set that maintains sorted order (e.g., `TreeSet`).
- **Hashing**: converting an object to an int hash value (`hashCode`).
- **Map**: key-value association.
- **SortedMap**: map with sorted keys (e.g., `TreeMap`).
- **Contract**: if `a.equals(b)` then `a.hashCode() == b.hashCode()` must hold.

7 Detailed Notes

7.1 Set vs List; HashSet vs TreeSet

List:

- ordered, duplicates allowed, index-based access.

Set:

- no duplicates, typically no index access.
- ideal for membership: "contains?" and deduplication.

HashSet:

- uses hashing; order is not guaranteed.
- fast average-case add/contains/remove.

TreeSet:

- keeps elements sorted (needs `Comparable` or a `Comparator`).
- slightly slower but provides sorted iteration.

7.2 Equality and hashing basics (must know)

Hash-based collections (`HashSet`, `HashMap`) use:

- `hashCode()` to choose a bucket,
- `equals()` to confirm identity inside that bucket.

Contract:

- If two objects are equal (`equals` returns true), their hash codes must be equal.
- If you override `equals`, you should also override `hashCode`.

Common mistakes:

- overriding only one of `equals`/`hashCode`,
- using mutable fields in keys (changing key after insertion breaks map/set behavior),
- using `==` instead of `equals` for strings.

7.3 Map basics; HashMap vs TreeMap

Map stores pairs: key → value. Common operations:

- `put(k, v)`, `get(k)`, `containsKey(k)`, `remove(k)`.

HashMap: no ordering, fast average-case lookup. **TreeMap:** keys sorted, good when you need ordered iteration or range queries.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

8 Running Project Connection

Use Set/SortedSet + Map/SortedMap to manage in-memory project data more cleanly and predictably inside Campus Resource Manager.

- Use Campus Resource Manager as the running case study, or map the same idea to your own project domain if you are using another example.
- Ask where today's idea should live: model, service, DAO, UI, or team workflow.

- Use the lecture example as a smaller version of the same design choice you will make in the capstone.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

9 Common Mistakes and Confusions

- Forgetting the equals/hashCode contract and blaming the collection.
- Expecting HashSet or HashMap to preserve sorted order.
- Using a map or set before confirming the problem truly needs keys or uniqueness.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

10 Worked Example: set + map counting

```java
import java.util.*;

public class Demo {
    public static void main(String[] args) {
        // Dedup using Set
        List<String> names = Arrays.asList("Asha", "Ravi", "Asha", "Neha");
        Set<String> unique = new HashSet<>(names);
        System.out.println("Unique: " + unique);

        // Counting with Map (frequency table)
        String[] words = {"a", "b", "a", "c", "b", "a"};
        Map<String, Integer> freq = new HashMap<>();
        for (String w : words) {
            freq.put(w, freq.getOrDefault(w, 0) + 1);
        }
        System.out.println("Freq: " + freq);

        // Sorted keys using TreeMap
        Map<String, Integer> sorted = new TreeMap<>(freq);
        System.out.println("Sorted freq: " + sorted);
    }
}
```

11 Demo / Observation Task

Use this block as a short reproducible demo or observation task.

- Reproduce the lecture's main example around *Set vs List; HashSet vs TreeSet* once without looking at the solution first.

- Change one input, rule, or design choice in Campus Resource Manager and predict the result before checking it.
- Record one success case, one edge case, and one failure case so the idea becomes observable instead of purely theoretical.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

12 Practice Check (with brief solutions)

1. When should you prefer a Set over a List?

   Answer: when duplicates are not allowed or you need fast membership checks.

2. Which collection keeps elements sorted: `HashSet` or `TreeSet`?

   Answer: `TreeSet`.

3. If you override `equals`, what else should you override?

   Answer: `hashCode`.

13 Self-Study Practice (no solutions)

1. Count character frequency in a string using `Map<Character, Integer>`.
2. Store student details in a map: rollNo → Student object.
3. Create a `TreeSet` of integers and print them in sorted order.

14 Further Study

- Revisit the prerequisite references if any recall bullet still feels weak.
- Rework the lecture example with one changed assumption, input, or design choice.
- Read the textbook and documentation references listed in the source line for a second explanation of the same topic.
- Apply the same idea once inside Campus Resource Manager so that the concept moves from theory to design practice.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

15 Exit Questions (with sample answers)

Q1. Why are `equals` and `hashCode` important for `HashSet`?

Sample answer: `HashSet` uses `hashCode` to locate a bucket and `equals` to check if an element already exists. Incorrect implementations can cause duplicates or missing lookups.

Q2. When do you choose `TreeMap` over `HashMap`?

Sample answer: Choose `TreeMap` when you need keys in sorted order or want ordered iteration/range queries. Use `HashMap` when ordering is not needed and you want faster average lookup.

Q3. When should TreeSet or TreeMap be preferred over hash-based versions?

Sample answer: When the requirement needs sorted traversal, natural ordering, or range-style behavior.